

Table of Contents

Week 3 — Summary & Examples Reference

Quick-Reference Card for Class Discussion

One-Line Definitions (Read These Aloud)

The Three Core Analogies

1. The Café Barista — Persistent vs Given vs Temporary
2. The Nametag Story — Why “My Own” Matters
3. Sokha’s Smartphone — State Changing Over Time

Eight Real-World Attribute & Method Examples

The COMMAND or QUERY Quick Test

Practice Set A — Easy

Practice Set B — Students Discuss

Three Kinds of Information — Quick Test

State-Change Scenarios — Use in Class Discussion

Scenario 1 — Water Bottle

Scenario 2 — Bank Balance Increase

Scenario 3 — Checking Without Changing

Scenario 4 — Two Objects, One Action

Scenario 5 — The Nametag Mistake in Action

Student Discussion Questions

Code Reading Guide — Not Syntax, Just Meaning

Common Student Confusions — Quick Fixes

Lesson Arc Summary

Homework Reminder

Week 3 — Summary & Examples Reference

Quick-Reference Card for Class Discussion

Use alongside: Week3_Attributes_Methods_State.md | **Class:** Wednesday, 17 June 2026 · 6:30 PM

One-Line Definitions (Read These Aloud)

Attributes are the pieces of information an object currently holds about itself.
Together, they form the object's state.

Methods are the actions an object can perform — its behaviors.

A Command method changes an object's state.
A Query method only reports the object's state — nothing changes.

A Parameter is information given to an action from outside, relevant only for that one action.

A Local/temporary value is created to help finish one action, then thrown away.

"My own" (this) means saving information given to you into your OWN permanent record — not just repeating it back.

The Three Core Analogies

1. The Café Barista — Persistent vs Given vs Temporary

Concept	Café Barista Story
Persistent attribute	"Drinks made today" — stays with her the whole shift

Local/temporary value	"I need 200ml of milk for this cup" — calculated once, then forgotten
Command method	<code>makeDrink()</code> — increases her drink count, changes her state
Query method	<code>checkDrinksMadeToday()</code> — just reports the number, changes nothing

2. The Nametag Story — Why “My Own” Matters

Concept	Nametag Story
Information given from outside	"Hi, my name is Sokha"
The mistake	Repeating "Sokha" back out loud, then forgetting it tomorrow
The fix	Saving "Sokha" into YOUR OWN contacts list
What <code>this</code> means	"Save this into MY OWN permanent record"
Why it matters	Without saving into your own record, the update never actually happens

3. Sokha's Smartphone — State Changing Over Time

Time	Action	State After
7:00 AM	Phone turned on	battery = 100%
7:30 AM	<code>makeCall()</code>	battery = 95%
10:00 AM	<code>takePhoto()</code> × 10	battery = 80%, photos = 10
1:00 PM	<code>playGame()</code>	battery = 45%
3:00 PM	<code>charge()</code>	battery = 100%
6:00 PM	<code>checkBattery()</code> ← query	battery = 100% (unchanged)

Eight Real-World Attribute & Method Examples

#	Object	Attributes (what it knows)	Query method	Command method
1	Smartphone	batteryPercent, isOn, storageUsed	checkBattery()	charge(), makeCall()
2	Bank account	balance, accountType	checkBalance()	deposit(), withdraw()
3	Tuk-tuk	fuelLevel, currentSpeed	checkFuelLevel()	refuel(), drive()
4	Student	currentGPA, creditsCompleted	checkGPA()	submitExam()
5	Water bottle	amountOfWaterInside	checkWaterLevel()	drinkSome(), refill()
6	Traffic light	currentColor	checkCurrentColor()	changeToNext()
7	Rice cooker	temperature, isOn	checkTemperature()	turnOn(), startCooking()
8	Library book	isAvailable	checkAvailability()	checkOut(), returnBook()

The COMMAND or QUERY Quick Test

Ask students this about any method:

"After this action runs, has ANYTHING about the object changed?"

- YES → **Command** method (it changes state)
- NO → **Query** method (it only reports state)

Practice Set A — Easy

Method	Answer	Why
<code>checkBalance()</code>	Query	Just tells you the number — balance stays the same

<code>checkBattery()</code>	Query	Just reports the percentage
<code>charge(30)</code>	Command	Battery percentage actually increases
<code>checkGPA()</code>	Query	Just reports the current GPA
<code>submitExam()</code>	Command	GPA is recalculated and updated

Practice Set B — Students Discuss

Method	Expected Answer
<code>checkFuelLevel()</code>	Query — just reports the fuel level
<code>refuel(10)</code>	Command — fuel level actually increases
<code>checkCurrentColor()</code> (traffic light)	Query — just reports red/yellow/green
<code>changeToNext()</code> (traffic light)	Command — color actually changes
<code>checkAvailability()</code> (library book)	Query — just reports true/false
<code>checkOut()</code> (library book)	Command — availability actually changes to false

Three Kinds of Information — Quick Test

For each item below, ask: “Does this last forever with the object, only for one action, or is it just a quick helper calculation?”

Information	Type	Why
A phone’s battery level	Persistent (attribute)	Belongs to the phone for its whole life
“Charge for 20 minutes” (told to the phone)	Given from outside (parameter)	Only relevant for this one charging action

A bank account's balance	Persistent (attribute)	Belongs to the account for its whole life
"Deposit \$200" (told to the account)	Given from outside (parameter)	Only relevant for this one deposit
"Fee = \$1 if over limit" (calculated mid-deposit)	Temporary (local)	Used once, then forgotten
A student's current GPA	Persistent (attribute)	Belongs to the student for their whole enrollment
"Score = 85 on this exam" (told to the record)	Given from outside (parameter)	Only relevant for this one exam submission

State-Change Scenarios — Use in Class Discussion

For each, ask: *"What action must have happened? Could this happen with no action at all?"*

Scenario 1 — Water Bottle

Object: Water bottle

Morning state: `amountOfWaterInside = full (500ml)`

Evening state: `amountOfWaterInside = empty (0ml)`

Question: What ACTION(S) must have happened?

Answer: Someone called `drinkSome()` multiple times, or poured it out.

Key idea: State NEVER changes without an action. Something happened.

Scenario 2 — Bank Balance Increase

Object: Sokha's BankAccount

Morning state: `balance = $500`

Evening state: `balance = $700`

Question: What action happened? What was the parameter?

Answer: `deposit()` was called. The parameter was the amount: \$200.

Key idea: Commands often need a parameter to know HOW MUCH to change.

Scenario 3 — Checking Without Changing

Object: Sokha's Smartphone

Action: Sokha looks at her screen and reads "battery: 67%"

Question: Did this action change the battery level?

Answer: No – `checkBattery()` is a query. Looking at a value never changes it. Only commands (like `charge()` or `makeCall()`) actually change state.

Scenario 4 — Two Objects, One Action

Objects: Sokha's BankAccount AND Dara's BankAccount

Action: Sokha calls `transfer(Dara's account, $100)`

Question: How many objects change state? What changes in each?

Answer: TWO objects change.

Sokha's account: balance decreases by \$100

Dara's account: balance increases by \$100

Key idea: One action (method call) can affect more than one object when it involves a parameter that is itself another object.

Scenario 5 — The Nametag Mistake in Action

Scenario: A school registrar updates a student's name.

The student says: "My name is Sokha Chenda" (this is given information).

MISTAKE: Registrar just says "Sokha Chenda" out loud and moves on.

→ Nothing is saved. The school record still shows the old name.

FIX: Registrar says: "Let me save THIS into the STUDENT'S OWN official record" → writes it into the permanent file.

→ Tomorrow, the correct name is still there.

Key idea: This is exactly what "`this.name = name`" means in code – save the given information into the object's OWN record.

Student Discussion Questions

Use these during the lesson or as exit-ticket questions:

1. "Think of an action in your daily life that is a QUERY (just reports something, changes nothing) and one that is a COMMAND (actually changes something). Share both."
2. "Why does an object's state never change by itself? What would go wrong if it could?"
(Expected: unpredictable behavior — your bank balance changing without any deposit or withdrawal would be alarming and untrustworthy.)
3. "In the nametag story, what would happen if you NEVER had your own contacts list at all — no permanent place to save anything? Why does an object need its own persistent attributes?"
4. "Give an example of 'temporary helper information' from your own daily life — something you calculate quickly in your head and then forget right after."
5. "If a teacher grades 30 students' exams one by one, is `gradeExam()` for student #1 affected by the temporary calculations the teacher did while grading student #5? Why not?"
(Expected: No — local/temporary values exist only during one specific action and don't carry over to the next.)

Code Reading Guide — Not Syntax, Just Meaning

Help students read these lines as concept descriptions, not programming instructions.

```
// Read as: "Every Smartphone object will know its brand and battery level"
String brand;
int batteryPercent;

// Read as: "This action takes input - how many minutes to charge"
void charge(int minutes) {

    // Read as: "A quick helper number, used once, then forgotten"
    int gained = minutes / 2;

    // Read as: "Update MY OWN battery level using that helper number"
    batteryPercent = batteryPercent + gained;
}

// Read as: "This action just reports the battery - changes nothing"
int checkBattery() {
    return batteryPercent;
}
```

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
brand, batteryPercent	Attributes – persistent state
minutes (parameter)	Given from outside, for this one action

gained, cost (local)	Temporary helper, used once, forgotten
charge(), makeCall()	Command methods – they CHANGE state
checkBattery()	Query method – it only READS state
this.batteryPercent	"MY OWN battery level" (the object's own copy)

Common Student Confusions — Quick Fixes

If a student says...	Respond with...
"Attributes and methods are the same"	"Attributes are what you KNOW (your mood, your battery level). Methods are what you DO (eat, charge, call). Different things."
"If I just say the new name out loud, it's saved"	"Like the nametag story — saying it isn't the same as writing it in YOUR OWN contacts. You must explicitly save it into your own record."
"Checking my balance changes my balance"	"Does looking at a thermometer change the temperature? No — checking (query) never changes anything. Only an action like depositing (command) does."
"My phone's battery just changes on its own"	"Does it? Or did you make a call, take a photo, or charge it? Something always causes the change — state never changes without an action."
"The temporary calculation should still be remembered later"	"Like the barista's milk measurement for one cup — it's used once and gone. Only things saved into a permanent attribute are remembered."

Lesson Arc Summary

WHAT THE LESSON BUILDS – STEP BY STEP

1. Start: "What do you currently know about yourself right now?"
→ Students describe ATTRIBUTES without knowing the term yet
2. Explain Attributes formally
→ Real-world examples table (phone, tuk-tuk, water bottle, student)

3. The Café Barista story
 - Persistent (drinks made today) vs given (customer order)
 - vs temporary (milk measurement)
 4. The Nametag story
 - Why "my own" (this) matters – saving vs just repeating
 5. "What can your phone DO?"
 - Students describe METHODS without knowing the term yet
 - Command vs Query – does it change anything or not?
 - Methods can take input (parameters) and give output (return values)
 - Methods can connect two objects (transfer, grade, serve)
 6. Sokha's Smartphone timeline
 - A full day of state changes, all caused by actions
 - A query event where nothing changes
 7. Java illustration
 - Code as concept, not syntax
 - Concept translation table
 8. Close: six key takeaways
 - Attributes = knows | Methods = does | Command changes, Query reads | State changes only through actions
-

Homework Reminder

No Java code required this week (optional bonus only).

Extend the `Smartphone` design from Week 2:

1. Add `storageUsedGB` attribute
2. Add `takePhoto()`, `deleteAllPhotos()` (commands), `checkStorage()` (query)
3. Write a 6-event timeline table showing state changes across one day, including one query event where nothing changes

Week 3 Summary Reference | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com